

International Cybersecurity and Digital Forensics Academy (ICDFA)

HUSSEINI SULEIMAN

2025/FWSD/11478

fwsd2511478@students.icdfa.edu.ng

DATE: 11TH September, 2026

SBT-DF203: Basic Networking Skills for Digital Forensics

LAB 3: SYN Flood Attack Investigation Using tshark

Three-Week Delivery Block 1/3 | 5-11 September 2026

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 3
Lab Title	SYN Flood Attack Investigation Using tshark
Delivery Block	1/3 of 3
Scheduled Dates	5-11 September 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	mySYNFloodCapture.pcap or a bounded loopback capture generated in the lab

1. Source Materials and Slide Alignment

- HTTP, tshark and SYN Flood: TCP three-way handshake, half-open connections, SYN flood concept, Scapy packet generation and tshark capture.
- The uploaded slide uses Apache on port 80, loopback capture and a four-packet controlled simulation.
- The assignment requires two authorized VMs or an equivalent isolated environment and evidence that distinguishes attack-like SYN activity from a normal handshake.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

An Apache service experienced repeated TCP SYN packets that were not followed by the final ACK of the three-way handshake. You will preserve the packet capture, establish a normal-traffic baseline, identify suspicious SYN patterns, calculate indicators, and conduct a strictly bounded loopback simulation to validate your interpretation.

3. Learning Outcomes

- Differentiate a complete TCP handshake from an incomplete or half-open attempt.
- Use tshark display filters and field extraction to identify SYN, SYN-ACK, ACK and reset packets.
- Calculate SYN counts, unique source ports, response behaviour and incomplete-handshake indicators.
- Create a four-packet loopback-only simulation with Scapy in an authorized environment.
- Develop a defensible timeline and incident finding from PCAP evidence.
- Recommend controls for detecting and mitigating SYN flood activity.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.

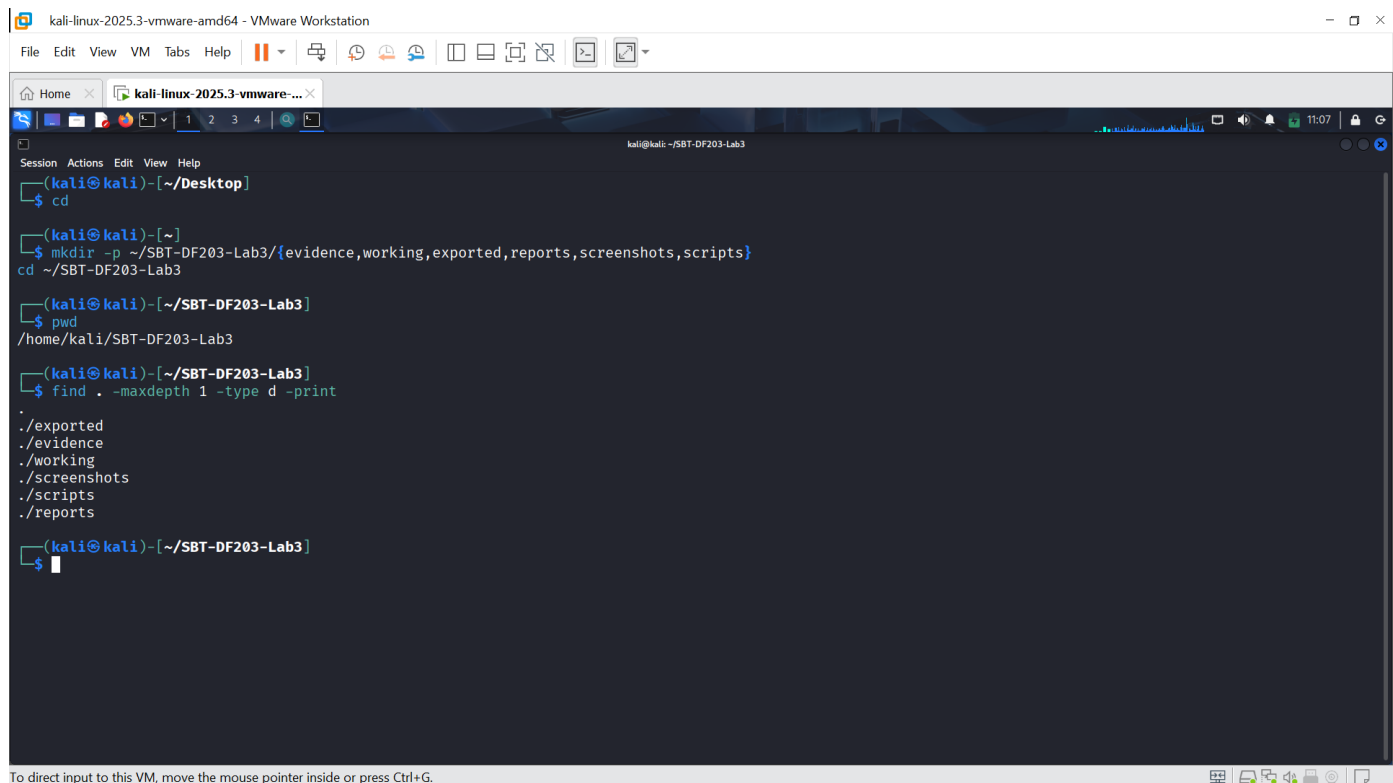
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The simulation is limited to four SYN packets against 127.0.0.1 or an instructor-assigned private lab IP. Do not increase packet count, automate continuous sending, spoof third-party addresses or target public systems.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux VM	Run tshark, Apache and Scapy.
Target service	Local Apache on 127.0.0.1:80	Safe target for bounded simulation.
Capture tool	tshark/Wireshark	Capture and analyze TCP flags.
Packet library	python3-scapy	Generate a small number of SYN packets.
Evidence	Provided PCAP or student capture	Forensic analysis source.
Optional second VM	Authorized host-only target	Demonstrate network capture without touching external networks.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-  
Lab3/{evidence,working,exported,reports,screenshots,scripts}  
cd ~/SBT-DF203-Lab3  
pwd  
find . -maxdepth 1 -type d -print
```



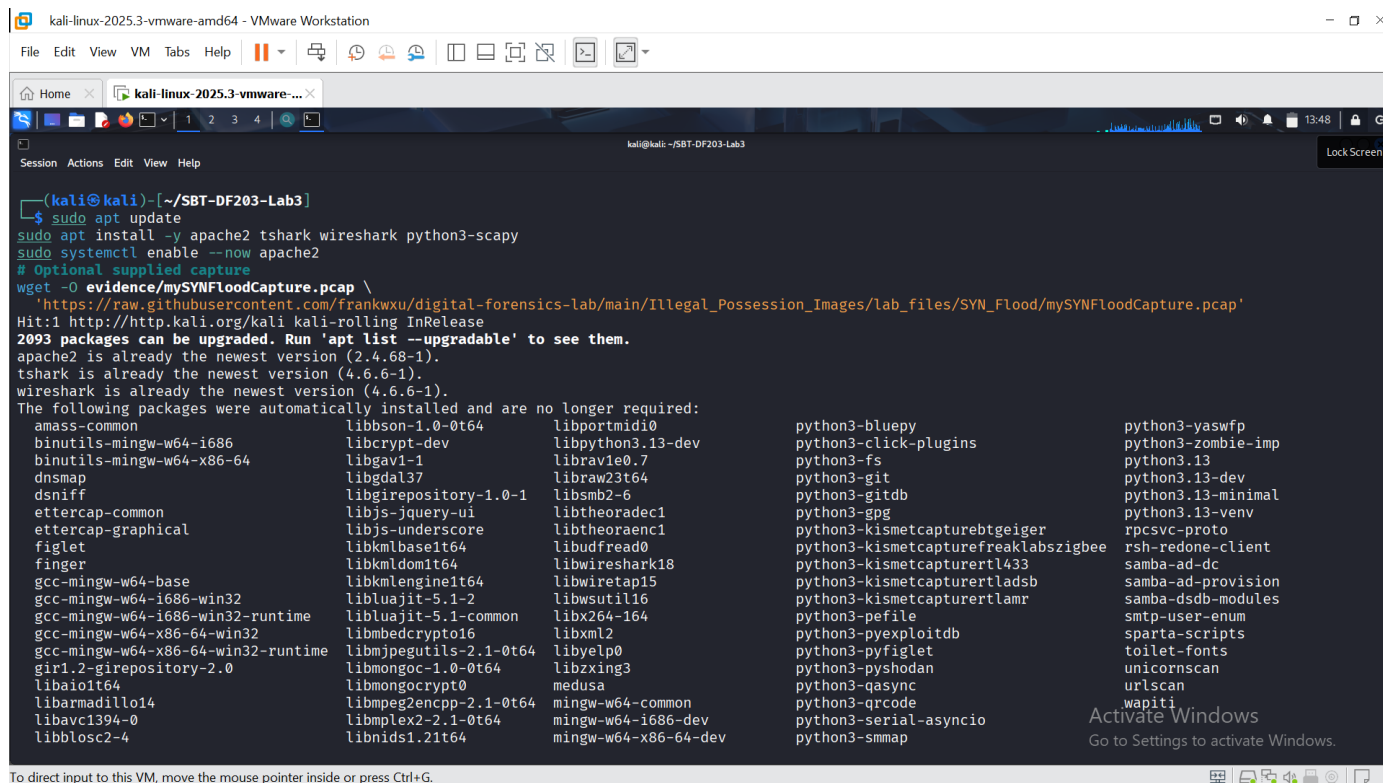
The screenshot shows a Kali Linux terminal window with the following commands and output:

```
kali@kali: ~/Desktop
$ cd
(kali@kali)~$
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
$ cd ~/SBT-DF203-Lab3
(kali@kali)~/SBT-DF203-Lab3$
$ pwd
/home/kali/SBT-DF203-Lab3
(kali@kali)~/SBT-DF203-Lab3$
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
(kali@kali)~/SBT-DF203-Lab3$
```

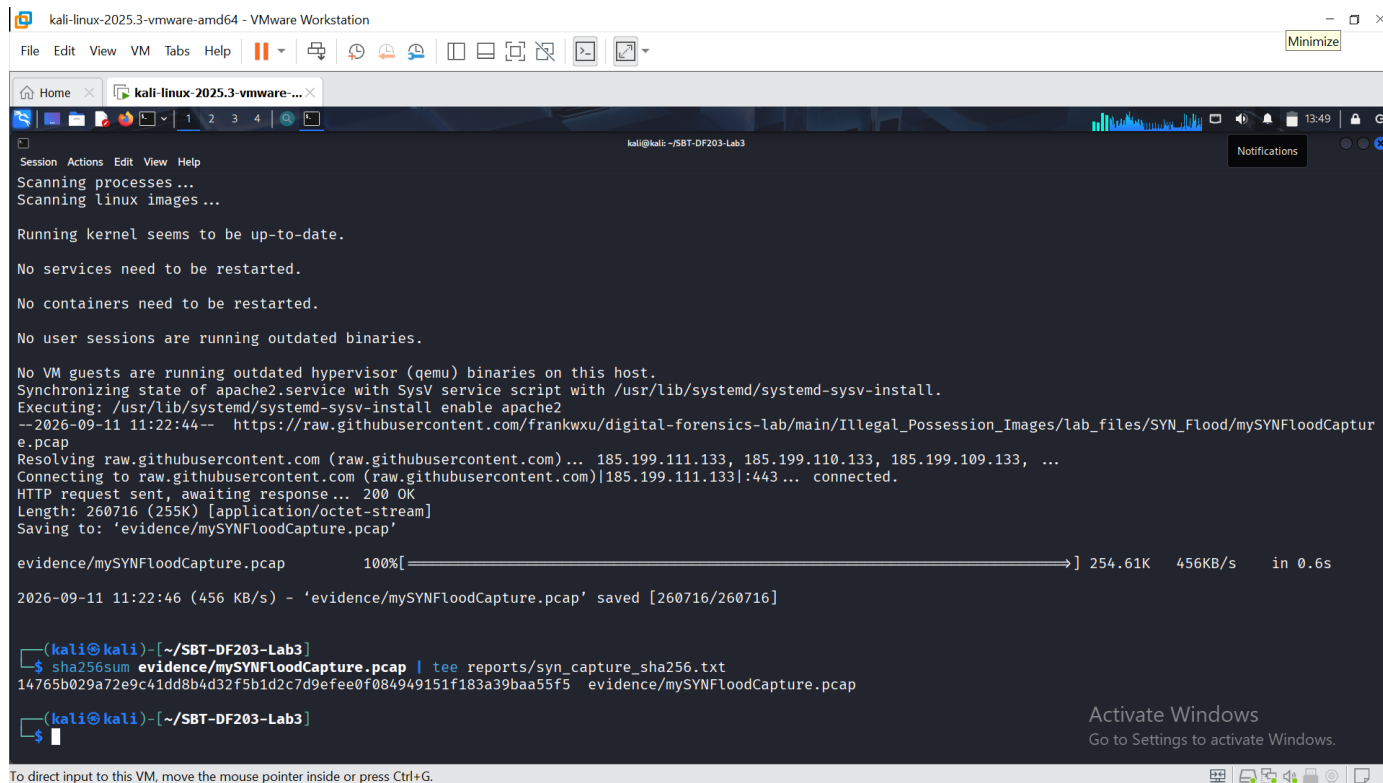
Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 tshark wireshark python3-scapy
sudo systemctl enable --now apache2
# Optional supplied capture
wget -O evidence/mySYNFloodCapture.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-
  lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.p
  cap'
sha256sum evidence/mySYNFloodCapture.pcap | tee
reports/syn_capture_sha256.txt
```

Screenshot required: The Apache service state, evidence file details and SHA-256 hash.



```
kali@kali: ~/SBT-DF203-Lab3
$ sudo apt update
$ sudo apt install -y apache2 tshark wireshark python3-scapy
$ sudo systemctl enable --now apache2
# Optional supplied capture
wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2093 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.68-1).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
The following packages were automatically installed and are no longer required:
  amass-common libbson-1.0-0t64 libportmidi0 python3-bluepy python3-yaswfp
  binutils-mingw-w64-i686 libcrypt-dev libpython3.13-dev python3-click-plugins python3-zombie-imp
  binutils-mingw-w64-x86_64 libgav1-1 librav1e0.7 python3-fs python3-zombie-imp
  dnsmasq libgdal37 libraw23t64 python3-git python3.13 python3.13
  dsniff libgirepository-1.0-1 librsmb2-6 python3-gitdb python3.13-dev python3.13-minimal
  ettercap-common libjs-jquery-ui libtheoraec1 python3-gpg python3.13-venv
  ettercap-graphical libjs-underscore libtheoraenc1 python3-kismetcapturebtgeiger rpcsvc-proto
  figlet libkmlbase1t64 libudfread0 python3-kismetcapturefreaklabszigbee rsh-redone-client
  finger libkmlcore1t64 libwireshark18 python3-kismetcaptureertl433 samba-ad-dc
  gcc-mingw-w64-base libkmlengine1t64 libwiretap15 python3-kismetcaptureertladsb samba-ad-provision
  gcc-mingw-w64-i686-win32 liblua5.1-2 libwsutil16 python3-kismetcaptureertlamlr samba-dsdb-modules
  gcc-mingw-w64-i686-win32-runtime liblua5.1-common libx264-164 python3-kefir sparta-scripts
  gcc-mingw-w64-x86_64-win32 libmbcncrypto16 libxml2 python3-kefir toilet-fonts
  gcc-mingw-w64-x86_64-win32-runtime libmjpegutils-2.1-0t64 libyelp0 python3-pyfiglet toilet-fonts
  gir1.2-girepository-2.0 libmongoc-1.0-0t64 libzxing3 python3-pyshodan unicornscan
  libaio1t64 libmjpegutils-2.1-0t64 medusa python3-qasync python3-qrcode urlscan
  libarmadillo14 libmpeg2encpp-2.1-0t64 mingw-w64-common python3-qrcode wapiti
  libavc1394-0 libmpeg2-2.1-0t64 mingw-w64-i686-dev python3-serial-asyncio
  libblosc2-4 libnids1.21t64 mingw-w64-x86_64-dev python3-smmap
```



```
kali@kali: ~/SBT-DF203-Lab3
$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f51d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap
$
```

7. Mini Evidence and Chain-of-Custody Worksheet

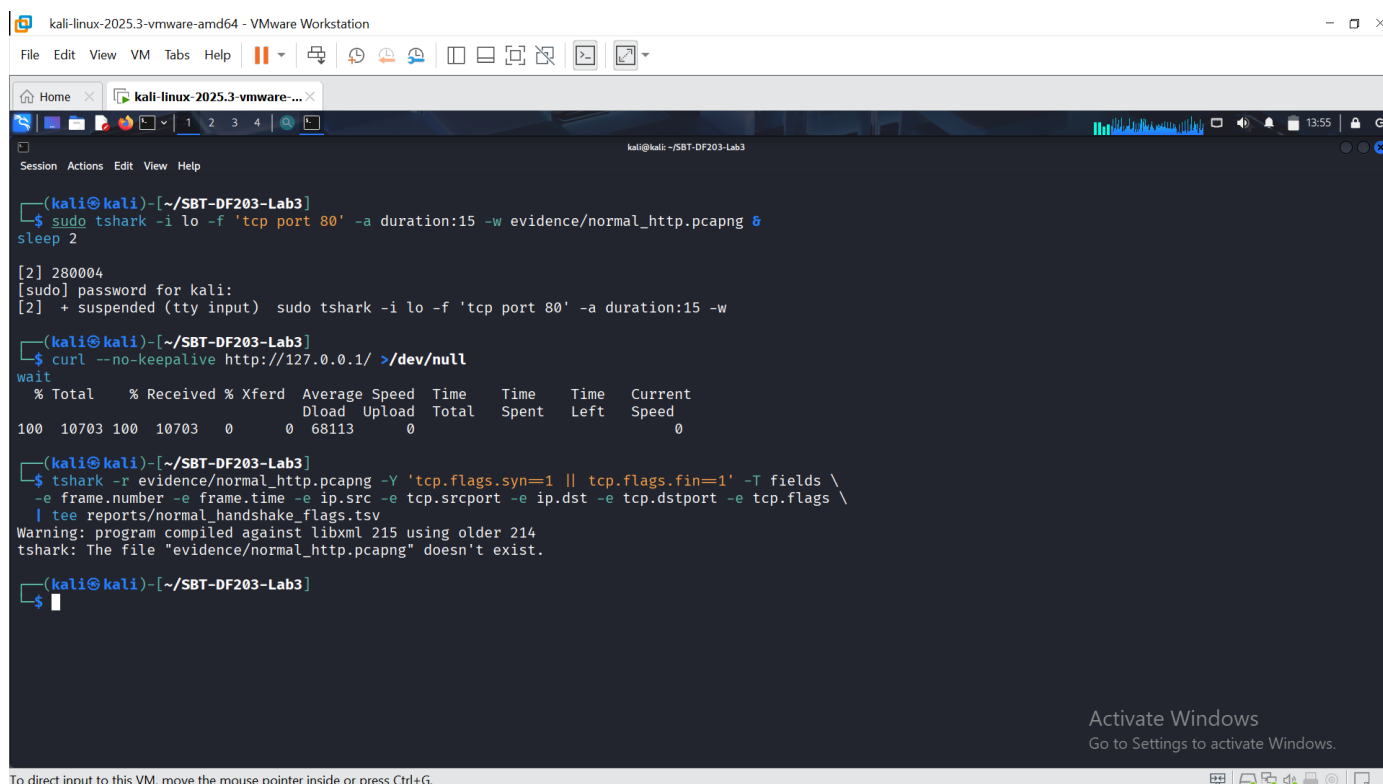
Field	Student Entry
Case/lab identifier	SBT-DF203-Lab3-Husseini Suleiman
Trainee name	Husseini Suleiman
Date and time started	2026-09-11 11:22:46 (456 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]
Evidence file name(s)	SBT-DF203-Lab3
Source or generation method	Kali terminal
Original SHA-256	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5
Working-copy SHA-256	de9262515fa24d8734944834fc4db2f32beef9ae69e32339e5f3cb176e0bffa5
Analysis workstation/V M	
Notes on any changes	

8. Part A - Establish a Normal Handshake Baseline

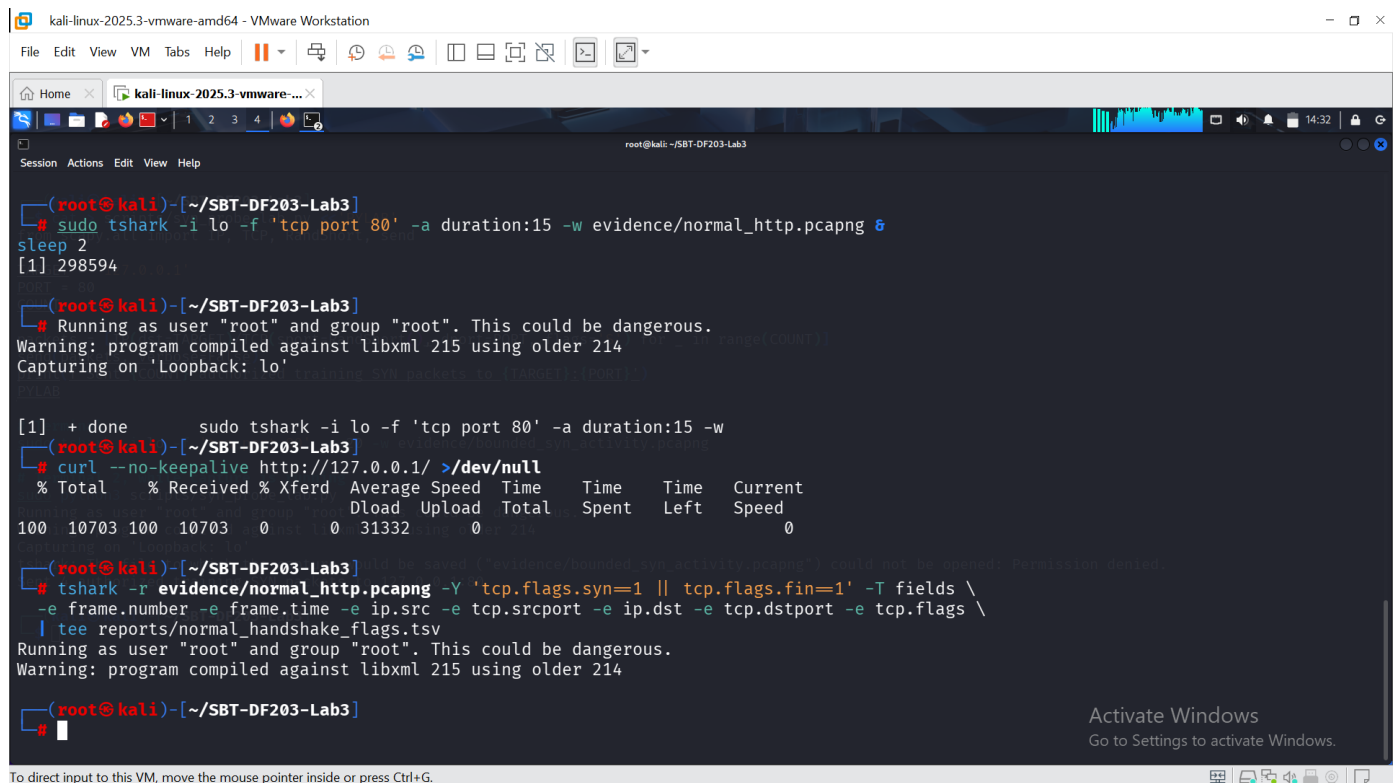
```
sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng
&
sleep 2
curl --no-keepalive http://127.0.0.1/ >/dev/null
wait

tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/normal_handshake_flags.tsv
```

Document the expected SYN, SYN-ACK and ACK sequence before examining suspicious traffic.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab3
(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
sleep 2
[2] 280004
[sudo] password for kali:
[2] + suspended (tty input) sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w
(kali@kali)~[~/SBT-DF203-Lab3]
$ curl --no-keepalive http://127.0.0.1/ >/dev/null
wait
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Total      %         0       0     0     0      0      0
100 10703 100 10703    0     0 68113    0      0      0
(kali@kali)~[~/SBT-DF203-Lab3]
$ tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/normal_handshake_flags.tsv
Warning: program compiled against libxml 215 using older 214
tshark: The file "evidence/normal_http.pcapng" doesn't exist.
(kali@kali)~[~/SBT-DF203-Lab3]
$
```



```
(root@kali)-[~/SBT-DF203-Lab3]
# sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
sleep 2
[1] 298594

(root@kali)-[~/SBT-DF203-Lab3]
# Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'

[1] + done      sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w
(root@kali)-[~/SBT-DF203-Lab3]
# curl --no-keepalive http://127.0.0.1/ >/dev/null
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
0    0     0    0         0     0      0      0      0      0
100 10703 100 10703    0     0    31332    0      0      0

(root@kali)-[~/SBT-DF203-Lab3]
# tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn=1 || tcp.flags.fin=1' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/normal_handshake_flags.tsv
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

(root@kali)-[~/SBT-DF203-Lab3]
#
```

9. Part B - Conduct the Bounded Loopback Simulation

Create the following script exactly. It sends four SYN packets to the local Apache service and then exits. It does not run continuously and does not target another organization.


```
cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

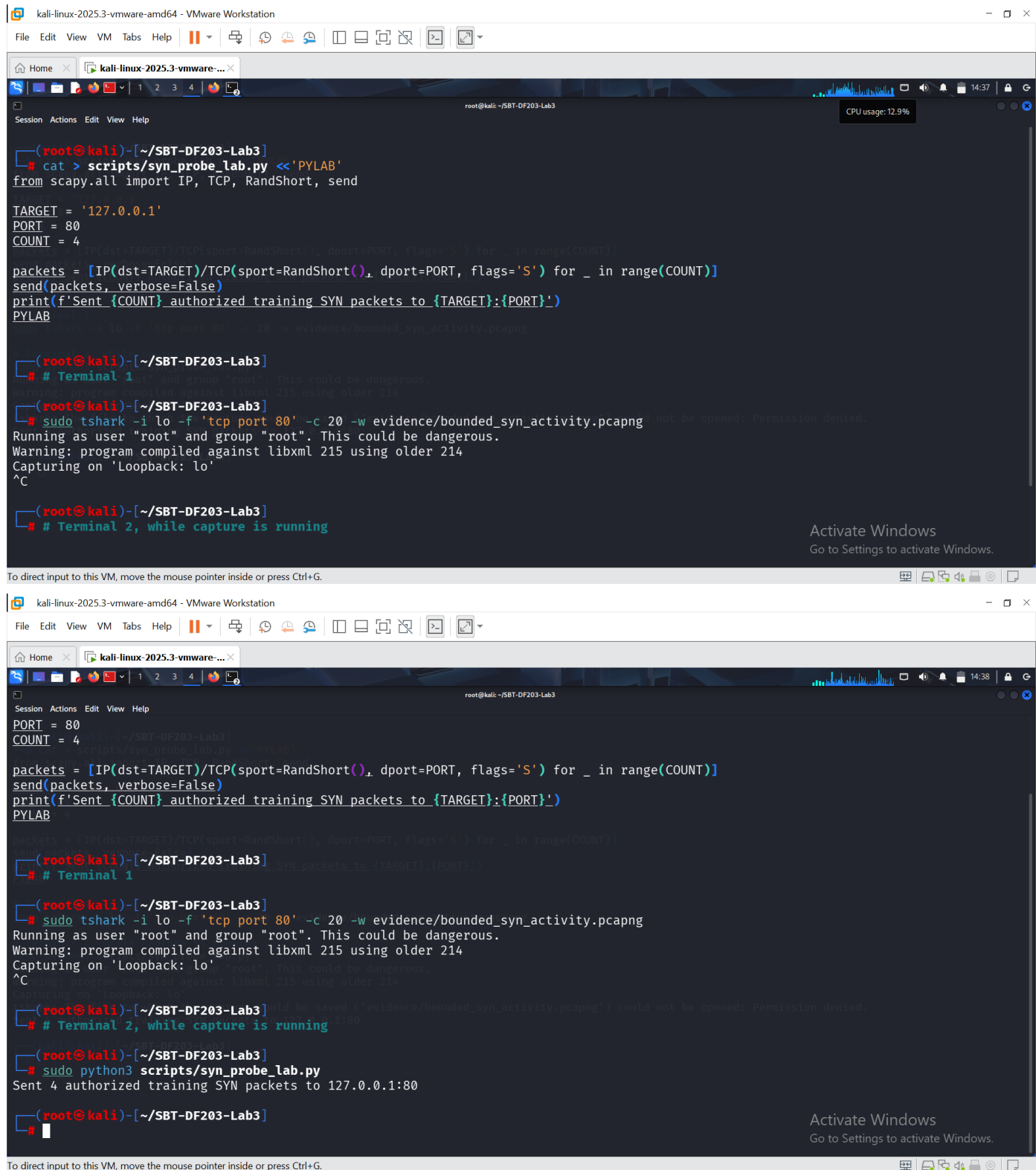
packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in
range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng

# Terminal 2, while capture is running
sudo python3 scripts/syn_probe_lab.py
```

Stop Condition

If the capture reaches 20 packets it stops automatically. Otherwise stop with Ctrl+C immediately after the four SYN packets and associated responses are captured.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation

File Edit View VM Tabs Help

kali-linux-2025.3-vmware-...x

root@kali: ~/SBT-DF203-Lab3
CPU usage: 12.9%

Session Actions Edit View Help

(root@kali)~[~/SBT-DF203-Lab3]
# cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

(root@kali)~[~/SBT-DF203-Lab3]
# # Terminal 1

(root@kali)~[~/SBT-DF203-Lab3]
# sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
^C

(root@kali)~[~/SBT-DF203-Lab3]
# # Terminal 2, while capture is running

Activate Windows
Go to Settings to activate Windows.

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

kali-linux-2025.3-vmware-amd64 - VMware Workstation

File Edit View VM Tabs Help

kali-linux-2025.3-vmware-...x

root@kali: ~/SBT-DF203-Lab3
CPU usage: 12.9%

Session Actions Edit View Help

PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

(root@kali)~[~/SBT-DF203-Lab3]
# # Terminal 1

(root@kali)~[~/SBT-DF203-Lab3]
# sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
^C

(root@kali)~[~/SBT-DF203-Lab3]
# # Terminal 2, while capture is running

(root@kali)~[~/SBT-DF203-Lab3]
# sudo python3 scripts/syn_probe_lab.py
Sent 4 authorized training SYN packets to 127.0.0.1:80

(root@kali)~[~/SBT-DF203-Lab3]
#

Activate Windows
Go to Settings to activate Windows.

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.
```

10. Part C - Identify SYN Indicators with tshark

```
PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee
reports/bounded_capture_hashes.txt
```

Initial SYNs only

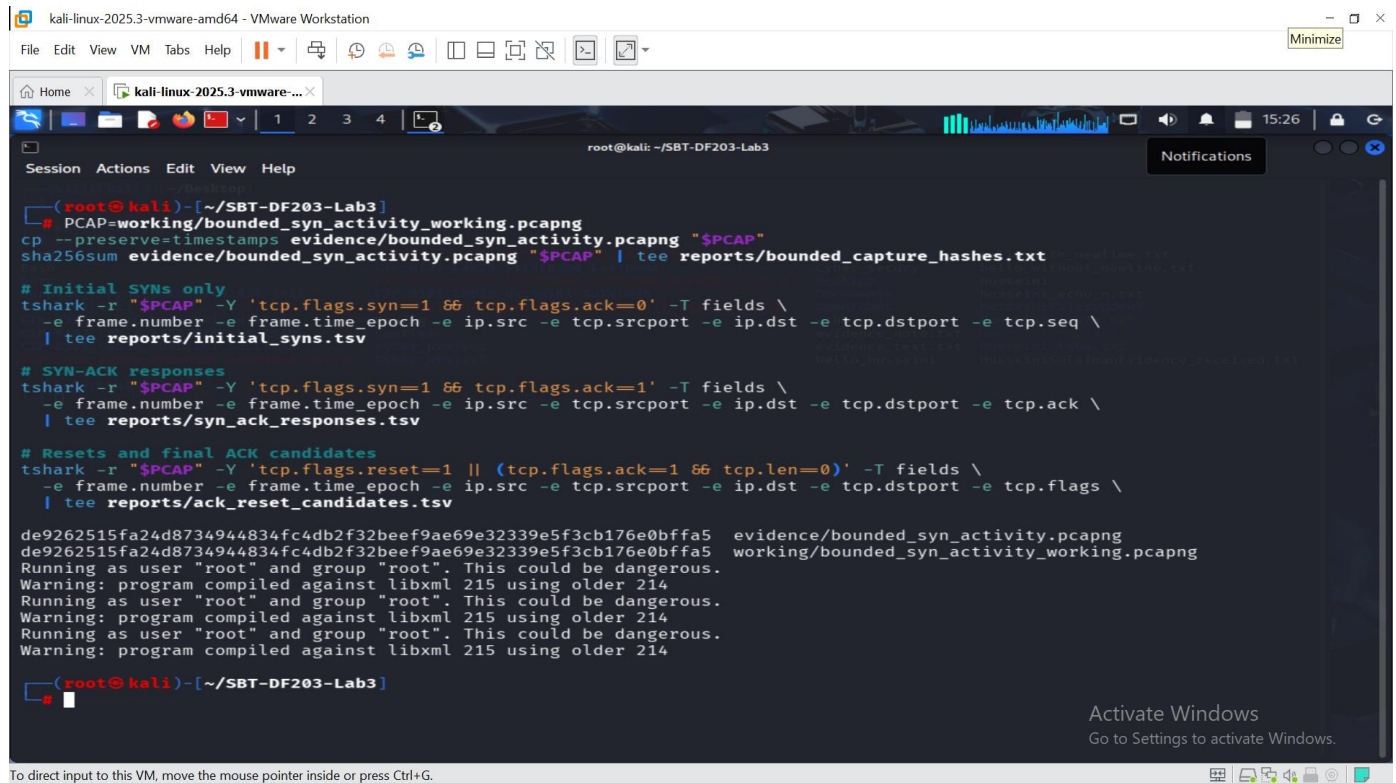
```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e
tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv
```

SYN-ACK responses

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e
tcp.dstport -e tcp.ack \
  | tee reports/syn_ack_responses.tsv
```

Resets and final ACK candidates

```
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T
fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e
tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv
```



The screenshot shows a terminal window in a Kali Linux VM. The user is root at a machine named 'kali'. The terminal displays several tshark commands for capturing network traffic related to a SYN flood attack. The commands are as follows:

```
(root@kali)-[~/SBT-DF203-Lab3]
# PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt

# Initial SYN's only
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
| tee reports/initial_syns.tsv

# SYN-ACK responses
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
| tee reports/syn_ack_responses.tsv

# Resets and final ACK candidates
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/ack_reset_candidates.tsv
```

Below the commands, the terminal shows the SHA256 hash of the PCAP file and a warning message:

```
de9262515fa24d8734944834fc4db2f32beef9ae69e32339e5f3cb176e0bffa5 evidence/bounded_syn_activity.pcapng
de9262515fa24d8734944834fc4db2f32beef9ae69e32339e5f3cb176e0bffa5 working/bounded_syn_activity_working.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
```

The terminal prompt is now (root@kali)-[~/SBT-DF203-Lab3].

At the bottom of the terminal window, there is a message: "Activate Windows. Go to Settings to activate Windows."

11. Part D - Quantify and Correlate the Activity

Counts by source and destination

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt
```

Unique client source ports

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt
```

Expert information and TCP analysis flags

```
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
```

```
tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment ||
tcp.analysis.duplicate_ack' \
  -T fields -e frame.number -e frame.time -e _ws.col.Info | tee
reports/tcp_analysis_events.tsv
```

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-...
root@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
-e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
| tee reports/syn_counts_by_pair.txt
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

(root@kali)-[~/SBT-DF203-Lab3]
# # Unique client source ports
tshark -r "$PCAP" -Y 'tcp.flags.syn=1 && tcp.flags.ack=0' -T fields -e tcp.srcport \
| sort -n | uniq | tee reports/unique_syn_source_ports.txt
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

(root@kali)-[~/SBT-DF203-Lab3]
# # Expert information and TCP analysis flags
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

(root@kali)-[~/SBT-DF203-Lab3]
# tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
-T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv

Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

(root@kali)-[~/SBT-DF203-Lab3]
#

```

Activate Windows
Go to Settings to activate Windows.

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

12. Part E - Compare Normal and Suspicious Sessions

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	4		
SYN-ACK count			
Completed handshakes	de9262515fa24d8734944834fc4db2f32beef9ae69e32339e5f3cb176e0bffa5 de9262515fa24d8734944834fc4db2f32beef9ae69e32339e5f3cb176e0bffa5		

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Unique client source ports			
HTTP request present?	http://127.0.0.1		
RST packets	4		
Observed duration			

A real SYN flood is defined by scale, rate, persistence and service impact. Four training packets are not a denial-of-service event; they reproduce the packet pattern in a safe way so that analysts can learn the indicators.

13. Detection and Mitigation Recommendations

- Alert on elevated SYN-to-completed-handshake ratios per source and destination.
- Monitor SYN backlog exhaustion, retransmissions, connection-state tables and service latency.
- Use SYN cookies, sensible backlog/time-out tuning, upstream rate limiting and DDoS protection where appropriate.
- Retain full packet capture or flow telemetry during incidents and synchronize system clocks.
- Correlate packet evidence with web server, firewall, load balancer and operating-system logs.

Required Screenshots Checklist

- ☐ Apache service and TCP port 80.
- ☐ Normal handshake in the baseline capture.

- ☐ Bounded Scapy script showing target and count.
- ☐ Initial SYN packet list.
- ☐ SYN-ACK response list.
- ☐ Any ACK or RST behaviour.
- ☐ Counts by source/destination.
- ☐ Unique source ports.
- ☐ Normal-versus-suspicious comparison table.
- ☐ Evidence hashes.

Student Report Template

1. Executive summary.
2. Authority, scope and safety controls.
3. Evidence description and hashes.
4. Normal handshake baseline.
5. Suspicious-traffic indicators and timeline.
6. Quantitative analysis.
7. Assessment of whether the capture proves a flood, scan or bounded simulation.
8. Detection and mitigation recommendations.
9. Appendix of commands and screenshots.

Submission Requirements

- One PDF report named SBT-DF203-Lab3_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
-----------------	-------	--------------------------------

Assessment Area	Marks	What the Instructor Will Check
Safety and evidence handling	15	Bounded loopback/host-only scope, hashes and clean stop.
Baseline analysis	15	Correct normal handshake identification.
tshark filtering	20	Accurate extraction of SYN, SYN-ACK, ACK/RST and fields.
Quantitative findings	20	Counts, ports, timing and correlation are correct.
Forensic interpretation	20	Clearly distinguishes packet pattern from proof of full DoS impact.
Report quality	10	Professional narrative and screenshots.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Initial SYN filter	<code>tcp.flags.syn==1 && tcp.flags.ack==0.</code>
SYN-ACK filter	<code>tcp.flags.syn==1 && tcp.flags.ack==1.</code>
Normal handshake	Final ACK follows SYN-ACK and application data may follow.
Suspicious pattern	Many initial SYNs with few completed handshakes.
Training limitation	Four packets demonstrate indicators but do not prove service denial.
Strong conclusion	Requires rate, duration, system impact and corroborating logs.

Command Reference Summary

Command	Purpose
tshark -r file -Y FILTER	Analyze a saved capture.
tcp.flags.syn	Filter on the SYN flag.
tcp.flags.ack	Filter on the ACK flag.
tcp.stream	Conversation identifier.
tshark -q -z expert	Show expert analysis.
sort uniq -c	Count repeated field combinations.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.